

AI ACADEMY, NATIONAL AI CENTER  
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

## Final Project Report

### Topic 4 – Async Research Assistant

---

|                 |                                                                                                                                         |              |                   |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------------|
| <b>Team:</b>    | Allahverdi & Isgandarli                                                                                                                 | <b>Date:</b> | 18 September 2026 |
| <b>Repo:</b>    | <a href="https://github.com/Yunis-Allahverdi/async-research-assistant">https://github.com/Yunis-Allahverdi/async-research-assistant</a> | <b>Tag:</b>  | v1.0-final        |
| <b>Members:</b> | <b>Yunis</b> <b>Allahverdi</b><br>(@Yunis-Allahverdi),<br><b>Fariz</b> <b>Isgandarli</b><br>(@farizisgenderli05-create)                 |              |                   |

---

## Executive Summary

We built an asynchronous research assistant: given a natural-language question, it queries Wikipedia, arXiv, and a web-search provider concurrently, then uses a provider-agnostic LLM to synthesize a single answer carrying inline [N] citations back to the retrieved sources. The provided `ai/` package (fetchers plus citation synthesizer) was treated as a frozen dependency; our work was the entire software-engineering layer around it — typed configuration, an `asyncio` orchestration layer, a filesystem cache, a retry / timeout / logging service wrapper, input validation, a CLI, an offline test suite, and a Docker image. The one number that proves the concurrency design works: on an identical three-source workload our benchmark drops wall-clock time from **11.43 s sequential to 3.64 s concurrent (3.1×)**. Our headline robustness story is that when *all three* live sources failed for reasons inside the frozen `ai/` layer, the system retried with backoff, hit its per-source timeouts independently, and degraded to a clean “no sources found” response rather than crashing. The most surprising lesson was that the hardest part of an “AI project” was not the AI at all but everything that makes a thin API robust, observable, and reproducible; if we started over we would push the retry policy to be token-aware and add a real HTTP-header shim layer we are permitted to own.

## Contents

|                                                |          |
|------------------------------------------------|----------|
| <b>Executive Summary</b>                       | <b>1</b> |
| <b>1 Project Overview</b>                      | <b>1</b> |
| 1.1 Problem framing & scope                    | 1        |
| 1.2 Provider choice and the AI module contract | 2        |
| <b>2 Architecture</b>                          | <b>2</b> |
| 2.1 Module map                                 | 2        |
| 2.2 OOP design & key abstractions              | 3        |
| 2.3 Data flow across module boundaries         | 3        |
| <b>3 Concurrency &amp; Performance</b>         | <b>3</b> |
| 3.1 Why this concurrency model                 | 3        |
| 3.2 Sequential-vs-concurrent benchmark         | 4        |
| <b>4 Robustness &amp; Error Handling</b>       | <b>4</b> |

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| 4.1      | Wrapping the AI module . . . . .                        | 4        |
| 4.2      | Failure-mode analysis (one concrete incident) . . . . . | 4        |
| 4.3      | Input validation . . . . .                              | 5        |
| 4.4      | Rate limit & politeness . . . . .                       | 5        |
| <b>5</b> | <b>Storage &amp; Caching</b>                            | <b>5</b> |
| 5.1      | Design . . . . .                                        | 5        |
| 5.2      | Caching policy . . . . .                                | 5        |
| <b>6</b> | <b>Testing</b>                                          | <b>6</b> |
| 6.1      | Strategy & coverage . . . . .                           | 6        |
| 6.2      | Notable tests . . . . .                                 | 6        |
| <b>7</b> | <b>Deployment &amp; Reproducibility</b>                 | <b>6</b> |
| 7.1      | Docker image . . . . .                                  | 6        |
| 7.2      | Configuration . . . . .                                 | 7        |
| <b>8</b> | <b>Limitations &amp; Future Work</b>                    | <b>7</b> |
| <b>9</b> | <b>Tools &amp; Acknowledgements</b>                     | <b>7</b> |
|          | <b>References</b>                                       | <b>7</b> |
| <b>A</b> | <b>Appendix — Reproducing the benchmark</b>             | <b>8</b> |

---

## 1. Project Overview

### 1.1 Problem framing & scope

The system answers a research question by gathering short, attributed excerpts from three independent sources and letting a language model fuse them into one concise, cited answer. The *input* is a single question string; the *outputs* are a synthesized answer plus a numbered reference list, where every [N] marker in the prose maps to one retrieved source. The only user-facing entry point is a command-line interface: `python -m researcher ask "..."`, with `--sources` to restrict the source set and `--no-cache` to bypass the cache. No HTTP server is required for this topic.

The provider stack is deliberately pluggable and selected entirely by environment variables: the LLM is provider-agnostic (Anthropic / OpenAI / Gemini via `LLM_PROVIDER`), and the web-search backend is one of Tavily / Serper / DuckDuckGo via `WEB_SEARCH_PROVIDER`. Wikipedia and arXiv need no key.

Relative to `TOPIC.md`, we implemented the full required SE layer and made two deliberate scope calls. We *tightened* the spec by making graceful degradation total: not only does a single failing source not abort the call, but a call in which *every* source fails still returns a structured result instead of an error. We *loosened* the optional persistence choice: rather than PostgreSQL we used a filesystem-JSON cache, which is persistent, inspectable, and adds no service to the container — an intentional simplicity-over-power trade-off for a pure key→blob workload.

### 1.2 Provider choice and the AI module contract

The LLM is selected at runtime through the provided factory; for development the default is `LLM_PROVIDER=anthropic` with model `claude-sonnet-4-6`, and the web backend default is `WEB_SEARCH_PROVIDER=duckduckgo` (no key, convenient for grading). Our code calls exactly four functions of the frozen package — `ai.fetch_wikipedia`, `ai.fetch_arxiv`, `ai.fetch_web`, and `ai.synthesize` — and the schemas `Source`, `Citation`, `AnswerWithCitations`. **We did not**

modify the public interface of the provided `ai/` package in any way. Against malformed model output we rely on two guards: the provided synthesizer already drops out-of-range / hallucinated citation indices, and our service layer validates that a call returned a usable, non-empty result before caching or returning it, treating provider faults as the transient `ProviderError` class described in §4.

## 2. Architecture

### 2.1 Module map

The system is a strict downward-dependency pipeline. The CLI drives the core researcher, which drives the concurrency orchestrator, which fans out to the service wrapper, which is the *only* module that touches the frozen `ai/` package, the cache, and the config.

```

    CLI  (src/cli.py, researcher/__main__.py)
        | ask "question" [--sources ...] [--no-cache]
        v
core/researcher.py          validate -> orchestrate -> synthesize ->
  ResearchResult
        |
        v
concurrency/orchestrator.py  asyncio.gather(return_exceptions=True)
                             + Semaphore(max_concurrency) + per-source asyncio.
                             timeout
                             | fetch_source(origin, query) x3, one shared httpx.AsyncClient
                             v
services/ai_service.py      retry+backoff | timeout | logging | cache lookup/
  store
        |           |           |
        v           v           v
storage/  ===AI BOUNDARY===  config.py
cache_store  ai/ (FROZEN)    Settings (env-driven)
(JSON+TTL)   fetch_* / synthesize

```

**Module-by-module.** `config.py` reads `.env` into a typed `Settings`. `models.py` holds our SE-layer schemas. `storage/cache_store.py` is a TTL-aware filesystem JSON cache. `services/ai_service.py` wraps every `ai.*` call with retry, timeout, logging, and caching — the single crossing of the AI boundary. `concurrency/orchestrator.py` runs the three fetches concurrently with bounded parallelism and graceful degradation. `core/researcher.py` validates input and assembles the final `ResearchResult`. `cli.py` parses arguments and renders the answer. Only the arrows into `ai_service` cross the AI boundary; swapping the LLM provider (Anthropic → OpenAI) changes *zero* of our files — it is one environment variable, because the provided factory hides the concrete provider behind an ABC.

### 2.2 OOP design & key abstractions

Our own defensible abstraction is *composition via a dispatch registry* in the service layer, rather than inheritance. The three source fetchers share one call shape, so instead of an `if/elif` ladder or a subclass per source we map `origin` → coroutine and inject the shared client and settings:

```

1 _FETCHERS = {
2     "wikipedia": fetch_wikipedia,
3     "arxiv": fetch_arxiv,
4     "web": fetch_web,
5 }
6
7 async def fetch_source(origin, query, *, client=None, settings, use_cache=True):
8     fetcher = _FETCHERS.get(origin)
9     if fetcher is None:

```

```

10         raise ValueError(f"unknown origin {origin!r}")
11     ...
12     return await _fetch_with_retry(fetcher, query, client=client, settings=settings
    )

```

**Justification.** We chose composition because the fetchers are stateless functions with an identical signature — there is no shared state or behaviour to inherit, so a class hierarchy would add ceremony without value. A registry keeps “add a source” to one line and keeps retry/timeout/caching in exactly one place. Where the problem genuinely called for an open provider set with per-provider state (API keys, endpoints), the *provided* code correctly used an ABC (`WebSearchProvider` with `Tavily/Serper/DuckDuckGo` subclasses); we deliberately did not duplicate that pattern where plain functions suffice.

## 2.3 Data flow across module boundaries

We defined three SE-layer models in `src/models.py`: `SourceResult` (per-source outcome: origin, sources, ok, error, elapsed), `ResearchResult` (question, answer, per-source results, degraded flag, total elapsed, from\_cache), and `CacheEntry` (key, sources, created\_at). We reused the provided `Source / AnswerWithCitations` verbatim wherever the data was exactly what `ai/` produced, and wrapped them only when we needed to attach SE concerns the provided schema does not carry — timing and success/failure for the orchestrator, cache metadata for the store. **No naked dictionaries cross module boundaries; every inter-module value is a typed pydantic model.**

## 3. Concurrency & Performance

### 3.1 Why this concurrency model

The workload is I/O-bound — three independent network round-trips that spend nearly all their wall-clock time waiting on remote servers, not on CPU. `asyncio` is therefore the right primitive: a single thread can keep all three requests in flight, and there is no CPU work to parallelize that would justify `ProcessPoolExecutor`, nor any blocking SDK that would force a `ThreadPoolExecutor`. Threads would also work for three requests but cost one OS thread per source and a shared connection pool is cleaner under one event loop. The bottleneck the design targets is *total network latency*: sequentially it is the sum of the three sources; concurrently it collapses to the slowest single source. We bound parallelism with `asyncio.Semaphore(max_concurrency)` (default 5, configurable via `MAX_CONCURRENCY`); at 1 the design degenerates to sequential, and at a very high value we would risk a provider rate-limit (HTTP 429), so a small bound respects politeness while still overlapping our three-way fan-out. The concurrent design starts winning at  $N \geq 2$  sources.

### 3.2 Sequential-vs-concurrent benchmark

`scripts/benchmark.py` runs the same query twice — once awaiting each source in turn, once through the orchestrator — always bypassing the cache so both do identical work. Measured on a Windows 11 laptop, Python 3.12.4, caches cleared:

| Workload                              | $N$ | Sequential (s) | Concurrent (s) | Speedup |
|---------------------------------------|-----|----------------|----------------|---------|
| 1 query, 3 sources (wiki, arxiv, web) | 3   | 11.43          | 3.64           | 3.1×    |

Command:

```
python -m scripts.benchmark "What is photosynthesis?" --sources wiki,arxiv,web
```

**Honest interpretation.** We reached almost exactly the theoretical 3× for a three-source fan-out, which is expected because the three latencies were comparable and the work is pure waiting. An important caveat: during measurement all three live endpoints were failing (see §4), so the numbers

time the concurrent-vs-sequential handling of the retry-and-timeout path rather than of successful downloads — but the comparison is still valid, because sequential pays each source's retry/timeout budget in turn (sum) while concurrent pays them in parallel (max). The new bottleneck after parallelizing the fetch is the *LLM synthesis step*, which is a single call that runs after all fetches complete and is therefore serial tail latency; pushing the curve further would mean streaming the synthesis or overlapping it with late-arriving sources.

## 4. Robustness & Error Handling

### 4.1 Wrapping the AI module

Every `ai.*` call goes through `ai_service`, which adds four things the thin provided layer lacks. **Retries:** a `tenacity` policy retries *only* `ProviderError` — the transient class (network errors, 5xx, 429) — for up to `MAX_RETRIES` (default 3) attempts with `wait_exponential` backoff. Non-transient failures such as a `ValueError` from empty input are *not* retried, because they cannot succeed on repeat and retrying them only wastes the budget. **Timeouts:** each attempt is wrapped in `asyncio.timeout(source_timeout)` (default 10 s) as a hard per-call ceiling. **Structured logging:** the standard `logging` module (never `print`) records origin, truncated query, result count, and elapsed time at INFO, full payloads at DEBUG, and each retry at WARNING; the level is set once from `LOG_LEVEL`. **Validation:** input is checked before the call (see §4.3) and the result is checked for usability before it is cached or returned. Walking one case end-to-end: a provider 429 is raised by `ai/` as a `ProviderError`, caught by the retry predicate, logged at WARNING, waited-out with exponential backoff, and retried; after the attempt budget is exhausted the original error is re-raised (`reraise=True`) and surfaces to the orchestrator, which records it as a failed `SourceResult` without cancelling the other sources.

### 4.2 Failure-mode analysis (one concrete incident)

**Incident: all three live sources unreachable.** We did not have to inject this failure — it occurred naturally, which made it a stronger test. Running `python -m researcher ask "How does CRISPR-Cas9 work?"` against the live network, every source failed for a reason inside the frozen `ai/` layer: arXiv returned HTTP 301 (the fetcher issues an `http://` request and does not follow the redirect to `https://`), Wikipedia returned HTTP 403 (the request carries no `User-Agent` header), and the pinned `duckduckgo-search` dependency had been renamed to `ddgs` upstream and now returns an empty list.

(i) *How it surfaced:* a normal live query. (ii) *What the system did:* each source was retried three times with backoff, each hit its timeout independently, and the orchestrator collected three failed `SourceResults` via `return_exceptions=True`. (iii) *What the user saw:* a clean message — “No sources could be found for this question...” followed by “*Note: wikipedia, arxiv, web could not be reached, so this answer may be incomplete*” — and no stack trace. (iv) *What the logs showed:* an ordered WARNING trail of `retrying (1/3)`, `(2/3)`, then per-source `FAILED` lines naming the exact HTTP status, which was enough to diagnose all three root causes without re-running the code. We originally expected at least DuckDuckGo (keyless) to succeed; the log is what revealed the upstream package rename. Because `ai/` is contractually frozen we could not patch the redirect/header/rename at the source, so the correct engineering response was exactly what the system already did — degrade cleanly and report — which validated the robustness path against real endpoints.

### 4.3 Input validation

The single entry point is the CLI. `validate_question` strips the question and rejects it if it is shorter than `MIN_QUESTION_LEN` (default 3) or longer than `MAX_QUESTION_LEN` (default 500), raising a `ValueError` that the CLI surfaces as a one-line message rather than a traceback. The `--sources` flag is validated against the allowed set `{wiki, arxiv, web}`; an unknown origin raises a clear error.

Environment values are validated by `pydantic-settings` at load time (wrong types fail fast). A 50,000-character query is rejected by the length bound before any network call, so an adversarial oversized input cannot reach a provider; cache keys are hashed, so a query can never escape the storage directory via crafted path characters.

#### 4.4 Rate limit & politeness

Politeness has two layers: the `Semaphore` bounds concurrent in-flight requests so we never burst more than `MAX_CONCURRENCY` (default 5) at once, and the exponential-backoff retry acts as a sleep-on-429 strategy — a provider 429 is a `ProviderError`, so the backoff naturally spaces out subsequent attempts. The highest burst our test suite produces is three simultaneous fetches (one per source), well under the bound. We did not observe a real 429 in development because the live endpoints failed earlier in the request path; the strategy is therefore exercised by unit tests that inject `ProviderError` rather than by a production 429.

### 5. Storage & Caching

#### 5.1 Design

We did not use SQLite/PostgreSQL. The workload is a pure key→value blob store ((`origin`, `query`) → list of `Sources`), for which a relational schema would be over-engineering and would add a service to the container. Instead each entry is one JSON file under `CACHE_DIR` (default `.cache/`), named by a stable key. The key is the source-of-truth of identity; the stored `Source` list and `created_at` timestamp are the cached payload.

```
1 def make_key(origin, query):
2     canonical = f"{origin.lower().strip()}|{query.lower().strip()}"
3     return f"{origin}_{hashlib.sha256(canonical.encode()).hexdigest()[:16]}"
```

The key is *canonicalised* (lower-cased, stripped) so “WHAT IS X?” and “what is x” collide onto one entry, then hashed so arbitrary query characters cannot break the filename or escape the directory.

#### 5.2 Caching policy

We cache each (`origin`, `query`) result for `CACHE_TTL` seconds (default 86,400 = 24 h). On read, the entry’s age is compared to the TTL and anything older is treated as a miss and refetched — this is time-based invalidation. `ai_service` checks the cache before every fetch and writes non-empty results after; `--no-cache` bypasses both ends. In a demo run that issues the same query twice, the first call is a miss and the second is a **100% cache hit** (the underlying fetcher is invoked exactly once, verified by a unit test that counts fetcher calls). The value most likely to go stale is an LLM-shaped result if the prompt template changes; in production we would add a prompt-version component to the key so a changed prompt misses the old entries automatically.

### 6. Testing

#### 6.1 Strategy & coverage

Every test runs **offline**: each `ai.*` call is replaced with a fake via `monkeypatch`, so the suite needs no network and no API keys and is safe in CI. Async code is driven by `pytest-asyncio` (`asyncio_mode = strict`). We chose not to chase the last few lines — three uncovered lines in `config.py` are the `__main__` print block, and the two in the orchestrator are a defensive branch that only fires on a malformed internal result — because covering them would test framework plumbing, not our logic.

| Suite                    | Coverage                   |
|--------------------------|----------------------------|
| src/ (our SE layer)      | 98 % (249 stmts, 6 missed) |
| Provided ai/ smoke tests | passing                    |

The full suite is **51 tests passing** in  $\sim 2$  s, and the provided `tests/test_ai_smoke.py` is unmodified and passes. A `mypy src` run reports 6 warnings: one is in the frozen `ai/` package (which we may not edit), and five are benign false positives in `ai_service.py` — `mypy` cannot prove the three registered fetchers share a signature, nor that the retry loop (`reraise=True`) always assigns before use. All are documented; none affects runtime, as the 51 passing tests confirm.

## 6.2 Notable tests

(i) **Happy path (end-to-end).** `test_fetch_source_happy_path` mocks a fetcher and asserts a well-formed `Source` flows back through the cache-write path. (ii) **Concurrency.** `test_orchestrator` verifies that when one of three gathered fetches raises, `return_exceptions=True` still yields three `SourceResults` — two `ok`, one `failed` — and no exception propagates. (iii) **Error path.** `test_fetch_source_retries_then_succeeds` and `..._reraises_after_exhaustion` pin the retry contract: a `ProviderError` that clears on the third attempt succeeds, while a permanent one is re-raised after exactly `MAX_RETRIES` attempts. `..._times_out` confirms a slow fetcher raises `TimeoutError` and is *not* retried. The test we wish we had time for is a load-style test asserting the semaphore never lets more than `MAX_CONCURRENCY` coroutines run at once under a large fan-out.

## 7. Deployment & Reproducibility

### 7.1 Docker image

```
docker build -t research-assistant .
docker run --rm research-assistant # runs: python demo_ai.py --offline
```

It is a **single-stage** build on `python:3.12-slim`. The image is **76.5 MB content** (324 MB on-disk including the shared base layers), comparable to a fresh `python:3.12-slim` ( $\sim 120$  MB) because our only added weight is the pinned Python dependencies and the small source tree; a `.dockerignore` keeps the `virtualenv`, `cache`s, `git` history, and `.env` out of the image. Dependencies are installed before the source is copied so the dependency layer is cached across code changes. The default command runs the offline demo end-to-end with no keys or network; the live CLI can be run by passing env vars with `-e / --env-file`.

### 7.2 Configuration

Every variable the app reads, and the effect if it is missing: `LLM_PROVIDER` (default `anthropic`); the provider's API key (`ANTHROPIC_API_KEY` etc.) — if missing, live synthesis raises a clear `ProviderError`, but the offline demo and tests are unaffected; `WEB_SEARCH_PROVIDER` (default `duckduckgo`); `TAVILY_API_KEY` / `SERPER_API_KEY` (needed only for those backends); `CACHE_DIR` (default `.cache`); `CACHE_TTL` (default 86400); `SOURCE_TIMEOUT` (default 10); `MAX_CONCURRENCY` (default 5); `MAX_RETRIES` / `RETRY_BACKOFF` (defaults 3 / 1.0); `MIN_QUESTION_LEN` / `MAX_QUESTION_LEN` (defaults 3 / 500); `LOG_LEVEL` (default `INFO`). Every non-key variable has a safe default, so the app runs with an empty `.env`.

## 8. Limitations & Future Work

- **Live retrieval depends on frozen code we cannot patch.** `arXiv` (301 redirect), `Wikipedia` (403 / missing `User-Agent`), and the renamed `duckduckgo-search` package all fail inside `ai/`; we

validate the pipeline via the offline demo and mocked tests. A permitted fix would be a thin request-shim layer of our own that adds headers and follows redirects *before* handing off, without editing `ai/`.

- **The filesystem cache has no concurrent-writer locking** and grows one file per unique query; a production version would add locking or move to SQLite/Redis and an eviction policy.
- **Synthesis is serial tail latency** — the single LLM call runs after all fetches, so it dominates once retrieval is parallel; streaming would help.
- **Rate limiting is request-count only**, not token-aware; a busy day with a real provider could still approach a tokens-per-minute limit the semaphore does not see.
- **Two-person scope** meant we prioritized the required layer over optional bonuses (multi-provider failover, cost telemetry, CI); these are the natural next week of work.

## 9. Tools & Acknowledgements

We used AI coding assistants as collaborators, and disclose this specifically rather than generically. **Claude** was used to scaffold the SE layer module by module and to explain each concept (`asyncio`, `tenacity` retry semantics, `pytest` mocking, Docker) as we built; every module was then reviewed, run, and unit-tested by its owner, and we can each defend our own files. Concrete example: the `ai_service` retry policy was drafted with Claude, but we tuned it after reading the live logs — narrowing the retry predicate to `ProviderError` only, so that non-transient `ValueErrors` are not uselessly retried — and we added the per-attempt `asyncio.timeout` after observing that a hung source would otherwise block within the retry budget. The provided `ai/` package, its schemas, and `demo_ai.py` are course-supplied and were left unmodified.

## References

- [1] Anthropic API documentation. <https://docs.anthropic.com/>
- [2] `tenacity` — retrying library. <https://tenacity.readthedocs.io/>
- [3] `pydantic-settings` documentation. [https://docs.pydantic.dev/latest/concepts/pydantic\\_settings/](https://docs.pydantic.dev/latest/concepts/pydantic_settings/)
- [4] `httpx` — async HTTP client. <https://www.python-httpx.org/>
- [5] `pytest-asyncio` documentation. <https://pytest-asyncio.readthedocs.io/>
- [6] Python `asyncio` — `gather`, `timeout`, `Semaphore`. <https://docs.python.org/3/library/asyncio.html>

## A. Appendix — Reproducing the benchmark

Anyone with the repo and a Python 3.11+ environment can reproduce the §3 table:

```
git clone https://github.com/Yunis-Allahverdi/async-research-assistant
cd async-research-assistant
python -m venv venv && source venv/bin/activate # Windows: venv\Scripts\Activate.ps1
python -m pip install -r requirements-ai.txt -r requirements.txt
python -m scripts.benchmark "What is photosynthesis?" --sources wiki,arxiv,web
```

*End of report — thank you for reading.*